

Ecole

백준 문제 풀이

2025.11.07

Python

타임머신으로 빨리 가기



핵심

문제[059] 타임머신으로 빨리 가기

시간 제한 1초 | 난이도 골드 IV | 백준 온라인 저지 11657번

N개의 도시와 한 도시에서 출발해 다른 도시에 도착하는 버스가 M개 있다. 각 버스는 A, B, C로 나타낼 수 있는데, A는 시작 도시, B는 도착 도시, C는 버스를 타고 이동하는 데 걸리는 시간이다. 시간 C가 양수가 아닐 때가 있다. C = 0일 경우에는 순간 이동을 할 때, C < 0일 경우에는 타임머신으로 시간을 되돌아갈 때다. 1번 도시에서 출발해 나머지 도시로 가는 가장 빠른 시간을 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 도시의 개수 N ($1 \leq N \leq 500$), 버스 노선의 개수 M ($1 \leq M \leq 6,000$)이 주어진다. 2번째 줄부터 M개의 줄에는 버스 노선의 정보 A, B, C ($1 \leq A, B \leq N$, $-10,000 \leq C \leq 10,000$)가 주어진다.

↑ 출력

만약 1번 도시에서 출발해 어떤 도시로 가는 과정에서 시간을 무한히 오래 전으로 되돌릴 수 있다면 1번째 줄에 -1을 출력한다. 그렇지 않다면 N - 1개 줄에 걸쳐 각 줄의 1번 도시에서 출발해 2번 도시, 3번 도시, ..., N번 도시로 가는 가장 빠른 시간을 순서대로 출력한다. 만약 이 도시로 가는 경로가 없다면 -1을 출력한다.

예제 입력 1

```
3 4 // 노드, 에지
1 2 4
1 3 3
2 3 -4
3 1 -2
```

예제 출력 1

```
-1
```

예제 입력 2

```
3 2
1 2 4
1 2 3
```

예제 출력 2

```
3
-1
```



1 에지 리스트에 에지 데이터를 저장한 후 거리 배열을 초기화한다.

최초 시작점에 해당하는 거리 배열값은 0으로 초기화한다.



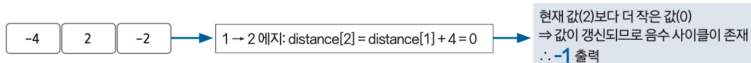
2 다음 순서에 따라 벨만-포드 알고리즘을 수행한다.

벨만-포드 알고리즘 수행 과정

- ① 모든 에지와 관련된 정보를 가져온 후 다음 조건에 따라 거리 배열의 값을 업데이트한다.
 - 출발 노드가 방문한 적이 없는 노드(출발 노드 거리 == INF)일 때 값을 업데이트하지 않는다.
 - 출발 노드의 거리 배열값 + 에지 가중치 < 종료 노드의 거리 배열값일 때 종료 노드의 거리 배열값을 업데이트한다.
- ② '노드 개수 - 1' 번만큼 ①을 반복한다.
- ③ 음수 사이클 유무를 알기 위해 모든 에지에 관해 다시 한번 ①을 수행한다. 이때 한 번이라도 값이 업데이트 되면 음수 사이클이 존재한다고 판단한다.

3 음수 사이클이 존재하면 -1, 존재하지 않으면 거리 배열의 값을 출력한다.

단, 거리 배열의 값이 INF일 경우에는 -1을 출력한다.



```
import sys
input = sys.stdin.readline
MAX = 1e9

n, m = map(int, input().split())
graph = [[] for _ in range(n+1)]
arr = [MAX]*(n+1)
arr[1] = 0
for _ in range(m):
    a, b, c = map(int, input().split())
    graph[a].append((b, c))
for i in range(n):
    for idx, temp in enumerate(graph):
        if idx == 0:
            continue
        for to_node, cost in temp:
            if arr[idx] != MAX and arr[idx] + cost < arr[to_node]:
                arr[to_node] = arr[idx] + cost
            if i == n-1:
                print(-1)
                exit()
for i in range(2, n+1):
    if arr[i] == MAX:
        print(-1)
    else:
        print(arr[i])
```


Java, JS

타임머신으로 빨리 가기



Java

```

public class P001 {
    static final long INF = Long.MAX_VALUE;

    // 이너 클래스로 Edge 정의 (static 필수)
    static class Edge {
        int u, v, w;

        Edge(int u, int v, int w) {
            this.u = u;
            this.v = v;
            this.w = w;
        }
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        int N = Integer.parseInt(st.nextToken());
        int M = Integer.parseInt(st.nextToken());

        List<Edge> edges = new ArrayList<>();
        long[] dist = new long[N + 1];
        Arrays.fill(dist, INF);

        for (int i = 0; i < M; i++) {
            st = new StringTokenizer(br.readLine());
            int u = Integer.parseInt(st.nextToken());
            int v = Integer.parseInt(st.nextToken());
            int w = Integer.parseInt(st.nextToken());
            edges.add(new Edge(u, v, w));
        }

        dist[1] = 0;

        // 벨만-포드 알고리즘
        for (int i = 1; i <= N; i++) {
            for (Edge edge : edges) {
                if (dist[edge.u] != INF && dist[edge.u] + edge.w < dist[edge.v]) {
                    dist[edge.v] = dist[edge.u] + edge.w;
                    // N번째 라운드에서도 값이 갱신된다면 음수 사이클 존재
                    if (i == N) {
                        System.out.println(-1);
                        return;
                    }
                }
            }
        }

        // 결과 출력
        StringBuilder sb = new StringBuilder();
        for (int i = 2; i <= N; i++) {
            if (dist[i] == INF) {
                sb.append("-1\n");
            } else {
                sb.append(dist[i]).append("\n");
            }
        }
        System.out.print(sb);
    }
}

```

JavaScript

```

const readline = require('readline');
const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout
});

let lines = [];

rl.on('line', (line) => {
    lines.push(line);
});

rl.on('close', () => {
    if (lines.length === 0) return;

    const [N, M] = lines[0].split(' ').map(Number);
    const edges = [];
    const dist = new Array(N + 1).fill(Infinity);

    for (let i = 1; i <= M; i++) {
        const [u, v, w] = lines[i].split(' ').map(Number);
        edges.push({ u, v, w });
    }

    dist[1] = 0;

    for (let i = 1; i <= N; i++) {
        for (let j = 0; j < M; j++) {
            const { u, v, w } = edges[j];
            if (dist[u] !== Infinity && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (i === N) {
                    console.log(-1);
                    return;
                }
            }
        }
    }

    for (let i = 2; i <= N; i++) {
        if (dist[i] === Infinity) {
            console.log(-1);
        } else {
            console.log(dist[i]);
        }
    }
});

```

Python

가장 빠른 버스 노선 구하기



핵심

문제[061] 가장 빠른 버스 노선 구하기

시간 제한 1초 | 난이도 골드 IV | 백준 온라인 저지 11404번

$n(2 \leq n \leq 100)$ 개의 도시가 있다. 그리고 한 도시에서 출발해 다른 도시에 도착하는 $m(1 \leq m \leq 100,000)$ 개의 버스가 있다. 각 버스는 한 번 사용할 때 필요한 비용이 있다. 모든 도시의 쌍 (A, B)에 관해 도시 A에서 B로 가는 데 필요한 비용의 최솟값을 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 도시의 개수 n , 2번째 줄에 버스의 개수 m 이 주어진다. 그리고 3번째 줄에서 $m + 2$ 줄까지 다음과 같은 버스의 정보가 주어진다. 먼저 처음에는 그 버스의 출발 도시의 번호가 주어진다. 버스의 정보는 버스의 시작 도시 a , 도착 도시 b , 한 번 타는 데 필요한 비용 c 로 이뤄져 있다. 시작 도시와 도착 도시가 같은 경우에는 없다. 비용은 100,000보다 작거나 같은 자연수다. 시작 도시와 도착 도시를 연결하는 노선은 1개가 아닐 수 있다.

↑ 출력

n 개의 줄을 출력해야 한다. i 번째 줄에 출력하는 j 번째 숫자는 도시 i 에서 j 로 가는 데 필요한 최소 비용이다. 만약, i 에서 j 로 갈 수 없을 때는 그 자리에 0을 출력한다.

예제 입력 1

```
5      // 도시 개수
14     // 노선 개수
1 2 2
1 3 3
1 4 1
1 5 10
2 4 2
3 4 1
3 5 1
4 5 3
3 5 10
3 1 8
1 4 2
5 1 7
3 4 2
5 2 4
```

예제 출력 1

```
0 2 3 1 4
12 0 15 2 5
8 5 0 1 1
10 7 13 0 3
7 4 10 6 0
```



Python

가장 빠른 버스 노선 구하기 - 정답

1 버스 비용 정보를 인접 행렬에 저장한다.

연결 도시가 같으면($i == j$) 0, 아니면 충분히 큰 수로 값을 초기화한다.

주어진 버스 비용 데이터 값을 인접 행렬에 저장한다.

최단 거리 배열 초기화

S \ E	1	2	3	4	5
1	0	∞	∞	∞	∞
2	∞	0	∞	∞	∞
3	∞	∞	0	∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

버스 비용
정보 저장

S \ E	1	2	3	4	5
1	0	2	3	1	10
2	∞	0	∞	2	∞
3	8	∞	0	1	1
4	∞	∞	∞	0	3
5	7	4	∞	∞	0

시작 도시와 도착 도시가 같으면 비용이 0

2 플로이드-워셜 알고리즘을 수행한다.

플로이드-워셜 점화식

// 두 도시의 연결 비용 중 최소값

$\text{Math.min}(\text{distance}[S][E], \text{distance}[S][K] + \text{distance}[K][E])$

알고리즘 수행 후 최단 거리 배열

S \ E	1	2	3	4	5
1	0	2	3	1	4
2	12	0	15	2	5
3	8	5	0	1	1
4	10	7	13	0	3
5	7	4	10	6	0

3 인접 행렬 자체가 모든 쌍의 최단 경로를 나타내는 정답 배열이다.

Python

```
import sys
INF = int(1e9)

n = int(sys.stdin.readline()) # 도시의 수
m = int(sys.stdin.readline()) # 버스의 수

graph = [[INF] * (n + 1) for _ in range(n + 1)] # 모든 최단 거리를 저장
for i in range(1, n + 1):
    for j in range(1, n + 1):
        if i == j:
            graph[i][j] = 0

for _ in range(m):
    a, b, c = map(int, sys.stdin.readline().split())
    graph[a][b] = min(c, graph[a][b]) # 노선이 하나가 아닐 수 있음 > 최소값 넣기

# 2. 다이나믹 프로그래밍
for k in range(1, n + 1):
    for a in range(1, n + 1):
        for b in range(1, n + 1):
            graph[a][b] = min(graph[a][b], graph[a][k] + graph[k][b])

for a in range(1, n + 1):
    for b in range(1, n + 1):
        if graph[a][b] == INF:
            print("0", end=" ")
        else:
            print(graph[a][b], end=" ")
    print()
```

Java, JS

가장 빠른 버스 노선 구하기



Java

```
public class P003 {
    static int[] parent;

    // 간선 정보를 저장할 클래스 (정렬을 위해 Comparable 구현)
    static class Edge implements Comparable<Edge> {
        int u, v, c;

        Edge(int u, int v, int c) {
            this.u = u;
            this.v = v;
            this.c = c;
        }

        @Override
        public int compareTo(Edge o) {
            return this.c - o.c; // 비용 기준 오름차순 정렬
        }
    }

    // Union-Find 알고리즘
    static int find(int x) {
        if (x == parent[x]) {
            return x;
        }
        // 경로 압축 (Path Compression)
        return parent[x] = find(parent[x]);
    }

    static void union(int x, int y) {
        x = find(x);
        y = find(y);

        if (x != y) {
            if (x <= y) {
                parent[y] = x;
            } else {
                parent[x] = y;
            }
        }
    }
}
```

```
public static void main(String[] args) throws IOException {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    StringTokenizer st = new StringTokenizer(br.readLine());

    int v = Integer.parseInt(st.nextToken());
    int e = Integer.parseInt(st.nextToken());

    List<Edge> edges = new ArrayList<>();
    for (int i = 0; i < e; i++) {
        st = new StringTokenizer(br.readLine());
        int a = Integer.parseInt(st.nextToken());
        int b = Integer.parseInt(st.nextToken());
        int c = Integer.parseInt(st.nextToken());
        edges.add(new Edge(a, b, c));
    }

    parent = new int[v + 1];
    for (int i = 1; i <= v; i++) {
        parent[i] = i;
    }

    long res = 0; // 결과값 (비용 합)

    // 간선을 최소 비용 순으로 오름차순 정렬
    Collections.sort(edges);

    // 크루스칼 알고리즘
    for (Edge edge : edges) {
        if (find(edge.u) != find(edge.v)) { // 부모 노드가 다름
            union(edge.u, edge.v); // 최소 신장 트리에 포함시킴
            res += edge.c;
        }
    }

    System.out.println(res);
}
```

JS

```
const readline = require('readline');
const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout
});

let input = [];
rl.on('line', (line) => {
    input.push(line);
});
rl.on('close', () => {
    if (input.length === 0) return;

    const n = parseInt(input[0]);
    const m = parseInt(input[1]);
    const INF = 1e9;
    const graph = Array.from(Array(n + 1), () => Array(n + 1).fill(INF));

    for (let i = 1; i <= n; i++) {
        graph[i][i] = 0;
    }

    for (let i = 2; i <= m + 2; i++) {
        const [a, b, c] = input[i].split(' ').map(Number);
        graph[a][b] = Math.min(graph[a][b], c);
    }

    // 플로이드-워셜 알고리즘
    for (let k = 1; k <= n; k++) {
        for (let a = 1; a <= n; a++) {
            for (let b = 1; b <= n; b++) {
                graph[a][b] = Math.min(graph[a][b], graph[a][k] + graph[k][b]);
            }
        }
    }

    let result = '';
    for (let a = 1; a <= n; a++) {
        for (let b = 1; b <= n; b++) {
            if (graph[a][b] === INF) {
                result += '0 ';
            } else {
                result += graph[a][b] + ' ';
            }
        }
        result += '\n';
    }
    console.log(result);
});
```

Python

최소 신장 트리 구하기



빈출

문제[064] 최소 신장 트리 구하기

시간 제한 2초 | 난이도 골드 IV | 백준 온라인 저지 1197번

최소 신장 트리(최소 스패닝 트리)는 주어진 그래프의 모든 노드들을 연결하는 부분 그래프 중 그 가중치의 합이 최소인 트리를 말한다. 그래프가 주어졌을 때 그 그래프의 최소 신장 트리를 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 노드의 개수 V ($1 \leq V \leq 10,000$)와 에지의 개수 E ($1 \leq E \leq 100,000$)가 주어진다. 다음 E 개의 줄에는 각 에지와 관련된 정보를 나타내는 세 정수 A, B, C 가 주어진다. 이는 A 번 노드와 B 번 노드가 가중치 C 인 에지로 연결돼 있다는 의미다. C 는 음수일 수도 있으며, 절댓값이 1,000,000을 넘지 않는다.

그래프의 노드는 1번부터 V 번까지 번호가 매겨져 있고, 임의의 두 노드 사이에 경로가 있다. 최소 신장 트리의 가중치가 -2,147,483,648보다 크거나 같고, 2,147,483,647보다 작거나 같은 데이터만 입력으로 주어진다.

↑ 출력

1번째 줄에 최소 신장 트리의 가중치를 출력한다.

예제 입력 1

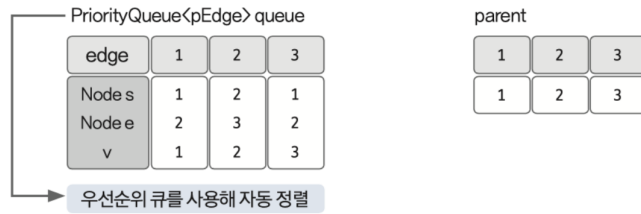
```
3 3 // 노드 개수, 에지 개수
1 2 1
2 3 2
1 3 3
```

예제 출력 1

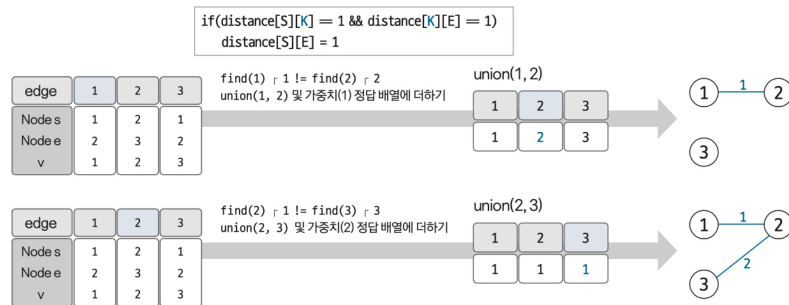
```
3
```



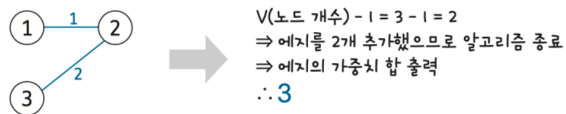
- 1 에지 리스트에 에지 정보를 저장한 후 부모 노드 데이터를 초기화한다.
사이클 생성 유무를 판단하기 위한 유니온 파인드용 부모 노드도 초기화한다.



- 2 크루스칼 알고리즘을 수행한다.
현재 미사용 에지 중 가중치가 가장 작은 에지를 선택하고, 연결했을 때 사이클의 발생 유무를 판단한다.
사이클이 발생하면 생략하고, 발생하지 않으면 에지값을 더한다.



- 3 과정 2에서 에지를 더한 횟수가 'V(노드 개수) - 1' 이 될 때까지 반복한다.
반복이 끝나면 에지의 가중치를 모두 더한 값을 출력한다.



Python

```
import sys
sys.setrecursionlimit(10**6)
input = sys.stdin.readline

v,e = map(int,input().split())
edges = [list(map(int,input().split())) for _ in range(e)] # a,b,c
parent = list(range(v+1))
res = 0

# 간선을 최소 비용 순으로 오름차순 정렬
edges.sort(key = lambda x: x[2])

# Union-Find 알고리즘
def find(x):
    if x == parent[x]:
        return x
    parent[x] = find(parent[x])
    return parent[x]

def union(x,y):
    x = find(x)
    y = find(y)

    if x <= y:
        parent[y] = x
    else:
        parent[x] = y

# 크루스칼 알고리즘
for i in range(e):
    x,y,c = edges[i]
    if find(x) != find(y): # 부모 노드가 다를
        union(x,y) # 최소 신장트리에 포함시킴
        res += c

print(res)
```

Java, JS

최소 신장 트리 구하기



Java

```

public class P003 {
    static int[] parent;

    // 간선 정보를 저장할 클래스 (정렬을 위해 Comparable 구현)
    static class Edge implements Comparable<Edge> {
        int u, v, c;

        Edge(int u, int v, int c) {
            this.u = u;
            this.v = v;
            this.c = c;
        }

        @Override
        public int compareTo(Edge o) {
            return this.c - o.c; // 비용 기준 오름차순 정렬
        }
    }

    // Union-Find 알고리즘
    static int find(int x) {
        if (x == parent[x]) {
            return x;
        }
        // 경로 압축 (Path Compression)
        return parent[x] = find(parent[x]);
    }

    static void union(int x, int y) {
        x = find(x);
        y = find(y);

        if (x != y) {
            if (x <= y) {
                parent[y] = x;
            } else {
                parent[x] = y;
            }
        }
    }
}

```

```

public static void main(String[] args) throws IOException {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    StringTokenizer st = new StringTokenizer(br.readLine());

    int v = Integer.parseInt(st.nextToken());
    int e = Integer.parseInt(st.nextToken());

    List<Edge> edges = new ArrayList<>();
    for (int i = 0; i < e; i++) {
        st = new StringTokenizer(br.readLine());
        int a = Integer.parseInt(st.nextToken());
        int b = Integer.parseInt(st.nextToken());
        int c = Integer.parseInt(st.nextToken());
        edges.add(new Edge(a, b, c));
    }

    parent = new int[v + 1];
    for (int i = 1; i <= v; i++) {
        parent[i] = i;
    }

    long res = 0; // 결과값 (비용 합)

    // 간선을 최소 비용 순으로 오름차순 정렬
    Collections.sort(edges);

    // 크루스칼 알고리즘
    for (Edge edge : edges) {
        if (find(edge.u) != find(edge.v)) { // 부모 노드가 다름
            union(edge.u, edge.v); // 최소 신장 트리에 포함시킴
            res += edge.c;
        }
    }

    System.out.println(res);
}

```

JS

```

const readline = require('readline');
const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
});

let lines = [];
rl.on('line', (line) => {
    lines.push(line);
});
rl.on('close', () => {
    if (lines.length === 0) return;

    const [v, e] = lines[0].split(' ').map(Number);
    let edges = [];
    for (let i = 1; i <= e; i++) {
        edges.push(lines[i].split(' ').map(Number)); // [a, b, c]
    }

    let parent = Array.from({ length: v + 1 }, (_, i) => i);
    let res = 0;

    edges.sort((a, b) => a[2] - b[2]);

    // Union-Find 알고리즘
    function find(x) {
        if (x === parent[x]) {
            return x;
        }
        return (parent[x] = find(parent[x]));
    }

    function union(x, y) {
        x = find(x);
        y = find(y);

        if (x !== y) {
            if (x <= y) {
                parent[y] = x;
            } else {
                parent[x] = y;
            }
        }
    }

    // 크루스칼 알고리즘
    for (let i = 0; i < e; i++) {
        const [x, y, c] = edges[i];
        if (find(x) !== find(y)) {
            // 부모 노드가 다름 (사이클 형성 X)
            union(x, y); // 최소 신장 트리에 포함시킴
            res += c;
        }
    }

    console.log(res);
});

```

Python

트리의 부모 찾기

**문제[067] 트리의 부모 찾기**

시간 제한 1초 | 난이도 실버 II | 백준 온라인 저지 11725번

루트 없는 트리가 주어진다. 이때 트리의 루트를 1이라고 정했을 때 각 노드의 부모를 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 노드의 개수 N ($2 \leq N \leq 100,000$), 2번째 줄부터 $N - 1$ 개의 줄에 트리상에 연결된 두 노드가 주어진다.

↑ 출력

1번째 줄부터 $N - 1$ 개의 줄에 각 노드의 부모 노드 번호를 2번 노드부터 순서대로 출력한다.

예제 입력 1

```
7 // 노드 개수
1 6
6 3
3 5
4 1
2 4
4 7
```

예제 출력 1

```
4
6
1
3
1
4
```

예제 입력 2

```
12
1 2
1 3
2 4
3 5
3 6
4 7
4 8
5 9
5 10
6 11
6 12
```

예제 출력 2

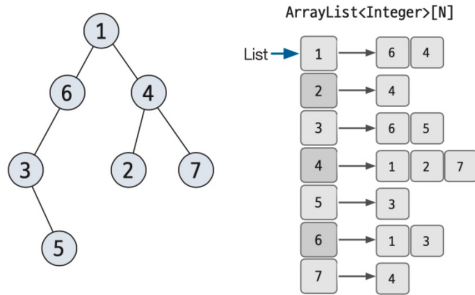
```
1
1
2
3
3
4
4
5
5
6
6
```



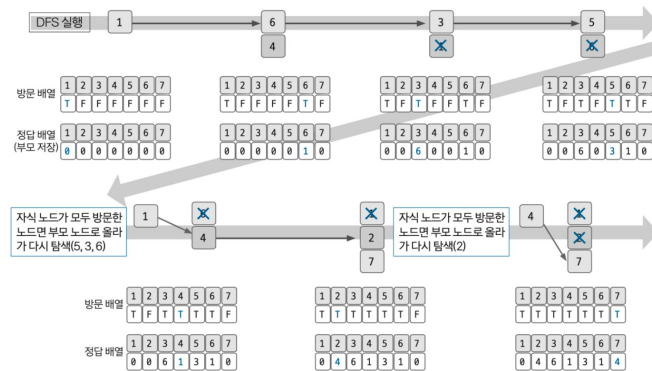
Python

트리의 부모 찾기 - 정답

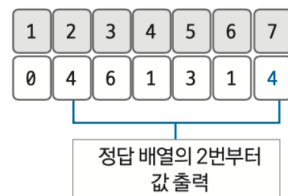
1 인접 리스트로 트리 데이터를 구현한다.



2 DFS 탐색을 수행한다. 수행할 때는 부모 노드의 값을 정답 배열에 저장한다.



3 정답 배열을 2번 인덱스부터 값을 차례대로 출력한다.



Python

```
import sys
sys.setrecursionlimit(10**6)
n = int(sys.stdin.readline())

graph = [[] for i in range(n+1)]

for i in range(n-1):
    a, b = map(int, sys.stdin.readline().split())
    graph[a].append(b)
    graph[b].append(a)

visited = [0]*(n+1)

arr = []

def dfs(s):
    for i in graph[s]:
        if visited[i] == 0:
            visited[i] = s
            dfs(i)

dfs(1)

for x in range(2, n+1):
    print(visited[x])
```

Java, JS

트리의 부모 찾기



Java

```
public class P004 {
    static ArrayList<Integer>[] graph;
    static int[] visited;

    public static void main(String[] args) throws IOException {
        // 입력 속도를 높이기 위해 BufferedReader 사용
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;

        int n = Integer.parseInt(br.readLine());

        // 그래프 초기화
        graph = new ArrayList[n + 1];
        for (int i = 1; i <= n; i++) {
            graph[i] = new ArrayList<>();
        }

        // 간선 정보 입력
        for (int i = 0; i < n - 1; i++) {
            st = new StringTokenizer(br.readLine());
            int a = Integer.parseInt(st.nextToken());
            int b = Integer.parseInt(st.nextToken());
            graph[a].add(b);
            graph[b].add(a);
        }

        visited = new int[n + 1];

        // 루트 노드 방문 처리 후 DFS 시작
        // 1번 노드의 부모는 없거나 자기 자신으로 표시하여 중복 방문 방지
        visited[1] = 1;
        dfs(1);

        // 결과 출력 (StringBuilder 사용으로 속도 향상)
        StringBuilder sb = new StringBuilder();
        for (int x = 2; x <= n; x++) {
            sb.append(visited[x]).append("\n");
        }
        System.out.print(sb);
    }

    // DFS 함수
    static void dfs(int s) {
        for (int next : graph[s]) {
            if (visited[next] == 0) {
                visited[next] = s; // 방문하지 않은 노드의 부모를 현재 노드(s)로 설정
                dfs(next);
            }
        }
    }
}
```

JS

```
const readline = require('readline');

const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
});

let lines = [];

rl.on('line', (line) => {
    lines.push(line);
});

rl.on('close', () => {
    if (lines.length === 0) return;

    const n = parseInt(lines[0]);
    const graph = Array.from({ length: n + 1 }, () => []);

    for (let i = 1; i < n; i++) {
        const [a, b] = lines[i].split(' ').map(Number);
        graph[a].push(b);
        graph[b].push(a);
    }

    const visited = new Array(n + 1).fill(0);

    // DFS 함수 (재귀)
    // JavaScript는 기본 재귀 깊이 제한이 있으므로,
    // 노드 수가 많으면 스택 오버플로우가 발생할 수 있습니다.
    // 필요 시 반복문 기반 DFS로 변경해야 할 수도 있습니다.
    function dfs(s) {
        for (const next of graph[s]) {
            if (visited[next] === 0) {
                visited[next] = s;
                dfs(next);
            }
        }
    }

    // 루트 노드 1부터 시작해서 부모 노드를 찾기 위해
    // visited[1]을 1로 설정하여 다시 방문하지 않도록 함
    visited[1] = 1;
    dfs(1);

    let result = '';
    for (let x = 2; x <= n; x++) {
        result += visited[x] + '\n';
    }
    console.log(result);
});
```

Python

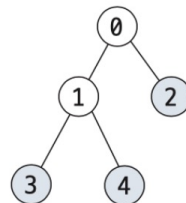
리프 노드의 개수 구하기



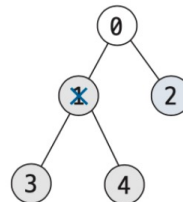
문제[068] 리프 노드의 개수 구하기

시간 제한 2초 | 난이도 실버 I | 백준 온라인 저지 1068번

트리에서 리프 노드는 자식의 개수가 0인 노드를 말한다. 예를 들어 다음과 같은 트리가 있다고 가정해 보자. 리프 노드의 개수는 3개(2, 3, 4)다.



노드를 지우면 그 노드와 노드의 모든 자손이 트리에서 제거된다. 예를 들어 1번 노드를 지우면 1번 노드의 자식 노드인 3, 4번 노드도 트리에서 제거된다. 리프 노드는 1개가 남는다.



주어진 트리에서 노드 1개를 지울 때 남은 트리에서 리프 노드의 개수를 구하는 프로그램을 작성하시오.

입력

1번째 줄에 트리의 노드의 개수 N 이 주어진다. N 은 50보다 작거나 같은 자연수다. 2번째 줄에 0번 노드부터 $N - 1$ 번 노드까지 각 노드의 부모가 주어진다. 만약 부모가 없다면 루트 노드 - 1이 주어진다. 3번째 줄에는 지울 노드의 번호가 주어진다.

출력

1번째 줄에 입력으로 주어진 트리에서 입력으로 주어진 노드를 지웠을 때 남아 있는 리프 노드의 개수를 출력한다.

예제 입력 1

```

9 // 노드 개수
-1 0 0 2 2 4 4 6 6
4 // 삭제 노드
  
```

예제 출력 1

```

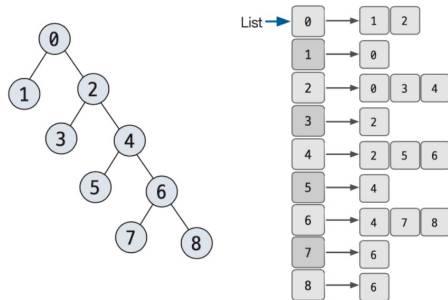
2
  
```



Python

리프 노드의 개수 구하기 - 정답

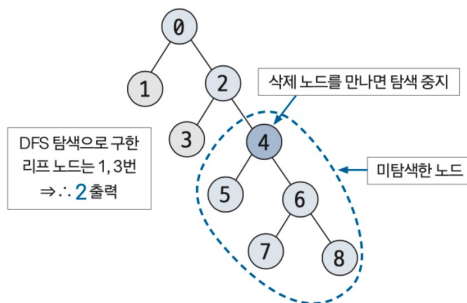
1 인접 리스트로 트리 데이터를 구현한다.



2 DFS 또는 BFS 탐색을 수행하면서 리프 노드의 개수를 센다.

단, 제거 대상 노드를 만났을 때는 그 아래 자식 노드들과 관련된 탐색은 중지한다.

이는 제거한 노드의 범위에서 리프 노드를 제거하는 효과가 있다.



Python

```

import sys
input = sys.stdin.readline

def dfs(num, arr):
    arr[num] = -2
    for i in range(len(arr)):
        if num == arr[i]:
            dfs(i, arr)

n = int(input())
arr = list(map(int, input().split()))
k = int(input())
count = 0

dfs(k, arr)
count = 0
for i in range(len(arr)):
    if arr[i] != -2 and i not in arr:
        count += 1
print(count)
  
```

Java, JS

리프 노드의 개수 구하기



Java

```

public class P005 {
    static int n;
    static int[] arr;
    static int k;

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        n = Integer.parseInt(br.readLine());
        arr = new int[n];

        StringTokenizer st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) {
            arr[i] = Integer.parseInt(st.nextToken());
        }

        k = Integer.parseInt(br.readLine());

        // k번 노드 삭제 수행
        dfs(k);

        int count = 0;
        for (int i = 0; i < n; i++) {
            // 삭제되지 않은 노드 중에서
            if (arr[i] != -2) {
                // i번 노드를 부모로 가지는 노드가 있는지 확인
                boolean isLeaf = true;
                for (int j = 0; j < n; j++) {
                    if (arr[j] == i) { // i를 부모로 가지는 노드 j가 존재하면 리프 노드 아님
                        isLeaf = false;
                        break;
                    }
                }
                // 자식이 없다면 리프 노드
                if (isLeaf) {
                    count++;
                }
            }
        }

        System.out.println(count);
    }

    // 삭제할 노드와 그 자손들을 -2로 표시하는 DFS 함수
    static void dfs(int num) {
        arr[num] = -2; // 현재 노드 삭제 표시
        for (int i = 0; i < n; i++) {
            // 현재 노드(num)를 부모로 가지는 노드(i)를 찾아 재귀 호출
            if (arr[i] == num) {
                dfs(i);
            }
        }
    }
}

```

JS

```

const readline = require('readline');

const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
});

let lines = [];

rl.on('line', (line) => {
    lines.push(line);
});

rl.on('close', () => {
    if (lines.length === 0) return;

    const n = parseInt(lines[0]);
    const arr = lines[1].split(' ').map(Number);
    const k = parseInt(lines[2]);

    // 삭제할 노드와 그 자손들을 -2로 표시하는 DFS 함수
    function dfs(num, arr) {
        arr[num] = -2; // 현재 노드 삭제 표시
        for (let i = 0; i < arr.length; i++) {
            // 현재 노드(num)를 부모로 가지는 노드(i)를 찾아 재귀 호출
            if (num === arr[i]) {
                dfs(i, arr);
            }
        }
    }

    // k번 노드 삭제 수행
    dfs(k, arr);

    let count = 0;
    for (let i = 0; i < arr.length; i++) {
        // 삭제되지 않은 노드 중에서
        if (arr[i] !== -2) {
            // i번 노드를 부모로 가지는 노드가 있는지 확인
            let isLeaf = true;
            for (let j = 0; j < arr.length; j++) {
                if (arr[j] === i) { // i를 부모로 가지는 노드 j가 존재하면 리프 노드 아님
                    isLeaf = false;
                    break;
                }
            }
            // 자식이 없다면 리프 노드
            if (isLeaf) {
                count++;
            }
        }
    }

    console.log(count);
});

```

Python

문자열 찾기

**문제[069] 문자열 찾기**

시간 제한 2초 | 난이도 실버 Ⅲ | 백준 온라인 저지 14425번

총 N개의 문자열로 이뤄진 집합 S가 있다. 입력으로 주어지는 M개의 문자열 중 집합 S에 포함돼 있는 것이 총 몇 개인지 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 문자열의 개수 N과 M($1 \leq N \leq 10,000$, $1 \leq M \leq 10,000$)이 주어진다. 그다음 N개의 줄에는 집합 S에 포함돼 있는 문자열이 주어지고, 그다음 M개의 줄에는 검사해야 하는 문자열이 주어진다. 입력으로 주어지는 문자열은 알파벳 소문자로만 이뤄져 있으며, 길이는 500을 넘지 않는다. 집합 S에 같은 문자열이 여러 번 주어지는 경우에는 없다.

↑ 출력

1번째 줄에 M개의 문자열 중 총 몇 개가 집합 S에 포함돼 있는지 출력한다.

예제 입력 1

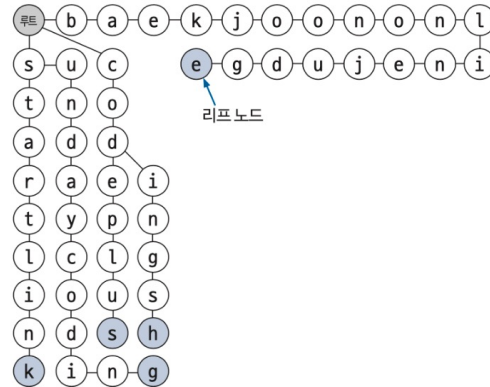
```
5 11 // N, M
baekjoononlinejudge
startlink
codeplus
sundaycoding
codingsh
baekjoon
codeplus
codeminus
startlink
starlink
sundaycoding
codingsh
codingsh
sondaycoding
startrink
icerink
```

예제 출력 1

```
4
```

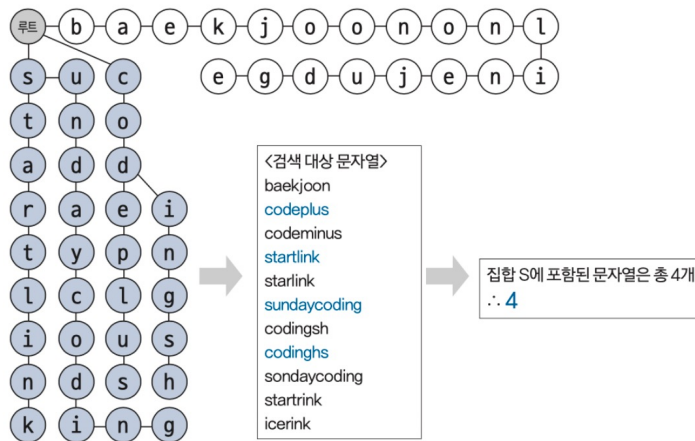



- 1 트라이 자료구조를 생성한다.
 문자열을 가리키는 위치의 노드가 공백이면
 신규 노드를 생성하고 아니라면 이동한다.
 문자열의 마지막에 도달하면 리프 노드라고 표시한다.



- 2 트라이 자료구조 검색으로 집합 S에 포함된 문자열을 센다.

대상 문자열을 검색했을 때 문자열이 끝날 때까지 공백 상태가 없고, 현재 문자의 마지막 노드가 트라이의 리프 노드라면 이 문자를 집합 S에 포함된 문자열로 센다.



Python

```
n, m = map(int, input().split())

n_string = set()
for i in range(n):
    n_string.add(input())

count = 0
for i in range(m):
    string = input()
    if string in n_string:
        count += 1

print(count)
```

Java, JS

문자열 찾기



Java

```
public class P006 {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        int n = Integer.parseInt(st.nextToken());
        int m = Integer.parseInt(st.nextToken());

        Set<String> nString = new HashSet<>();

        // N개의 문자열을 HashSet에 추가
        for (int i = 0; i < n; i++) {
            nString.add(br.readLine());
        }

        int count = 0;
        // M개의 문자열이 HashSet에 존재하는지 확인
        for (int i = 0; i < m; i++) {
            if (nString.contains(br.readLine())) {
                count++;
            }
        }

        System.out.println(count);
    }
}
```

JS

```
const readline = require('readline');

const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
});

let lines = [];

rl.on('line', (line) => {
    lines.push(line);
});

rl.on('close', () => {
    if (lines.length === 0) return;

    const [n, m] = lines[0].split(' ').map(Number);
    const nString = new Set();

    let currentLine = 1;
    // N개의 문자열을 Set에 추가
    for (let i = 0; i < n; i++) {
        nString.add(lines[currentLine++]);
    }

    let count = 0;
    // M개의 문자열이 Set에 존재하는지 확인
    for (let i = 0; i < m; i++) {
        if (nString.has(lines[currentLine++])) {
            count++;
        }
    }

    console.log(count);
});
```

Python

트리 순회하기



문제[070] 트리 순회하기

시간 제한 2초 | 난이도 실버 I | 백준 온라인 저지 1991번

이진 트리를 입력받아 전위 순회

```
preorder traversal
```

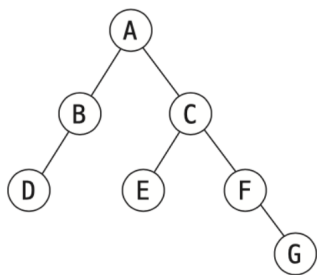
, 중위 순회

```
inorder traversal
```

, 후위 순회

```
postorder traversal
```

한 결과를 출력하는 프로그램을 작성하시오.



예를 들어 위와 같은 이진 트리가 입력되면 다음과 같은 결과가 나온다.

- 전위 순회한 결과: ABDCEFG // (루트) (왼쪽 자식) (오른쪽 자식)
- 중위 순회한 결과: DBAECFG // (왼쪽 자식) (루트) (오른쪽 자식)
- 후위 순회한 결과: DBEGFCA // (왼쪽 자식) (오른쪽 자식) (루트)

↓ 입력

1번째 줄에 이진 트리의 노드 개수 N ($1 \leq N \leq 26$), 2번째 줄부터 N 개의 줄에 걸쳐 각 노드와 그의 왼쪽 자식 노드, 오른쪽 자식 노드가 주어진다. 노드의 이름은 A부터 차례대로 영문자 대문자로 매겨지며, 항상 A가 루트 노드가 된다. 자식 노드가 없을 때는 .으로 표현된다.

↑ 출력

1번째 줄에 전위 순회, 2번째 줄에 중위 순회, 3번째 줄에 후위 순회한 결과를 출력한다. 각 줄에 N 개의 알파벳을 공백 없이 출력하면 된다.

예제 입력 1

```

7 // 노드 개수
A B C
B D .
C E F
E . .
F . G
D . .
G . .
  
```

예제 출력 1

```

ABDCEFG
DBAECFG
DBEGFCA
  
```



1 2차원 배열에 트리 데이터를 저장한다.

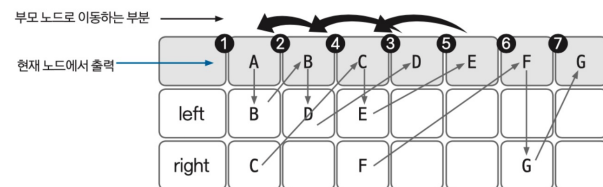
실제로 저장할 때는 코드에서 적절하게 index화 하여 저장한다.

	A	B	C	D	E	F	G
left	B	D	E				
right	C		F			G	

2 전위 순회 함수를 구현해 실행한다.

전위 순회 순서

현재 노드 → 왼쪽 노드 → 오른쪽 노드 순서로 탐색



3 중위 순회, 후위 순회 함수도 과정 2와 같은 방식으로 구현해 실행한다.

중위 순회 순서

왼쪽 노드 → 현재 노드 → 오른쪽 노드 순서로 탐색

후위 순회 순서

왼쪽 노드 → 오른쪽 노드 → 현재 노드 순서로 탐색

Python

```
import sys

N = int(sys.stdin.readline().strip())
tree = {}

for n in range(N):
    root, left, right = sys.stdin.readline().strip().split()
    tree[root] = [left, right]

def preorder(root):
    if root != '.':
        print(root, end='') # root
        preorder(tree[root][0]) # left
        preorder(tree[root][1]) # right

def inorder(root):
    if root != '.':
        inorder(tree[root][0]) # left
        print(root, end='') # root
        inorder(tree[root][1]) # right

def postorder(root):
    if root != '.':
        postorder(tree[root][0]) # left
        postorder(tree[root][1]) # right
        print(root, end='') # root

preorder('A')
print()
inorder('A')
print()
postorder('A')
```

Java, JS

트리 순회하기



Java

트리 순회하기 - 정답

Java

```

public class P007 {
    // 파이션의 tree = {} 와 동일한 역할을 하는 Map
    // key: 루트 노드 이름, value: [왼쪽 자식, 오른쪽 자식] 배열
    static Map<String, String[]> tree = new HashMap<>();
    static StringBuilder sb = new StringBuilder();

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int N = Integer.parseInt(br.readLine().trim());

        for (int i = 0; i < N; i++) {
            StringTokenizer st = new StringTokenizer(br.readLine());
            String root = st.nextToken();
            String left = st.nextToken();
            String right = st.nextToken();
            tree.put(root, new String[]{left, right});
        }

        preorder("A");
        sb.append("\n");
        inorder("A");
        sb.append("\n");
        postorder("A");

        System.out.println(sb.toString());
    }

    // 전위 순회 (Root -> Left -> Right)
    static void preorder(String root) {
        if (root.equals(".")) return;
        sb.append(root);
        preorder(tree.get(root)[0]);
        preorder(tree.get(root)[1]);
    }

    // 중위 순회 (Left -> Root -> Right)
    static void inorder(String root) {
        if (root.equals(".")) return;
        inorder(tree.get(root)[0]);
        sb.append(root);
        inorder(tree.get(root)[1]);
    }

    // 후위 순회 (Left -> Right -> Root)
    static void postorder(String root) {
        if (root.equals(".")) return;
        postorder(tree.get(root)[0]);
        postorder(tree.get(root)[1]);
        sb.append(root);
    }
}

```

JS

```

const readline = require('readline');
const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
});

let lines = [];

rl.on('line', (line) => {
    lines.push(line);
});

rl.on('close', () => {
    if (lines.length === 0) return;

    const N = parseInt(lines[0]);
    const tree = {};

    for (let i = 1; i <= N; i++) {
        const [root, left, right] = lines[i].trim().split(' ');
        tree[root] = [left, right];
    }

    let result = '';

    // 전위 순회 (Preorder): Root -> Left -> Right
    function preorder(root) {
        if (root === '.') return;
        result += root;
        preorder(tree[root][0]);
        preorder(tree[root][1]);
    }

    // 중위 순회 (Inorder): Left -> Root -> Right
    function inorder(root) {
        if (root === '.') return;
        inorder(tree[root][0]);
        result += root;
        inorder(tree[root][1]);
    }

    // 후위 순회 (Postorder): Left -> Right -> Root
    function postorder(root) {
        if (root === '.') return;
        postorder(tree[root][0]);
        postorder(tree[root][1]);
        result += root;
    }

    preorder('A');
    result += '\n';
    inorder('A');
    result += '\n';
    postorder('A');

    console.log(result);
});

```


감사합니다!